RELIABILITY ENGINEERING, FAILURE RECOVERY, AND RESILIENT SYSTEM DESIGN

Engineering Case Study for Business-Critical Data, Messaging, Document, and AI Workflows

Author	Venkata Lakshmi Parimala Pasupuleti
Primary domains	Affordable Housing, backend systems, data engineering, production AI
Systems represented	Income limits, eligibility, RabbitMQ, documents, migrations, LeaseI, AgentProbe
Document type	Technical engineering case study
Implementation boundary	Production experience plus clearly labeled target patterns
Core principle	Recover to a known, validated business state - not merely a running process
Status	Portfolio documentation based on real engineering situations

This document does not claim ownership of a company-wide site reliability program. It consolidates reliability practices demonstrated through systems I built, owned, supported, or designed, and separates direct experience from modernization recommendations.

Document Map

1. Executive Summary
2. Reliability Context and Scope
3. Reliability Principles
4. Failure Taxonomy
5. Reliability Architecture Layers
6. Workflow State and Recovery
7. Database Reliability
8. ETL and Data Publication Reliability
9. RabbitMQ and Asynchronous Processing
10. Idempotency and Duplicate Prevention
11. Retry, Timeout, and Backoff Strategy
12. Dead-Letter and Poison-Message Handling
13. API and Service Resilience
14. External Dependency Failure
15. Document Generation and Storage Recovery
16. Release, Migration, and Rollback Reliability
17. Configuration and Feature-Flag Reliability
18. Graceful Degradation and Partial Availability
19. Human Escalation and Operational Control
20. AI Runtime Reliability
21. AI Evaluation and Regression Protection
22. Observability and Reliability Metrics
23. Testing Failure and Recovery
24. Incident Recovery Examples
25. Reliability Trade-offs
26. Modernization Roadmap
27. Reusable Reliability Checklists
28. Lessons Learned

1. Executive Summary

Reliability engineering is the discipline of designing systems that continue to produce trustworthy outcomes when components fail, data is incomplete, workloads spike, dependencies slow down, or humans make mistakes. In business-critical Affordable Housing systems, reliability cannot be reduced to uptime. The system must preserve data integrity, select the correct effective rules, prevent duplicate financial effects, generate complete documents, and provide a controlled path to recovery.

My production experience across annual income-limit imports, RabbitMQ processing, document migration, database rollout, AWS S3 integration, and support incidents showed that resilient systems need more than retries. They need explicit workflow state, idempotency, validation, isolation, rollback, reconciliation, observability, and human escalation. A process that restarts successfully but publishes the wrong data is not reliable.

This case study presents a reliability model that connects infrastructure behavior with business correctness. It documents proven practices from production ownership and clearly marks target improvements for a more mature enterprise architecture.

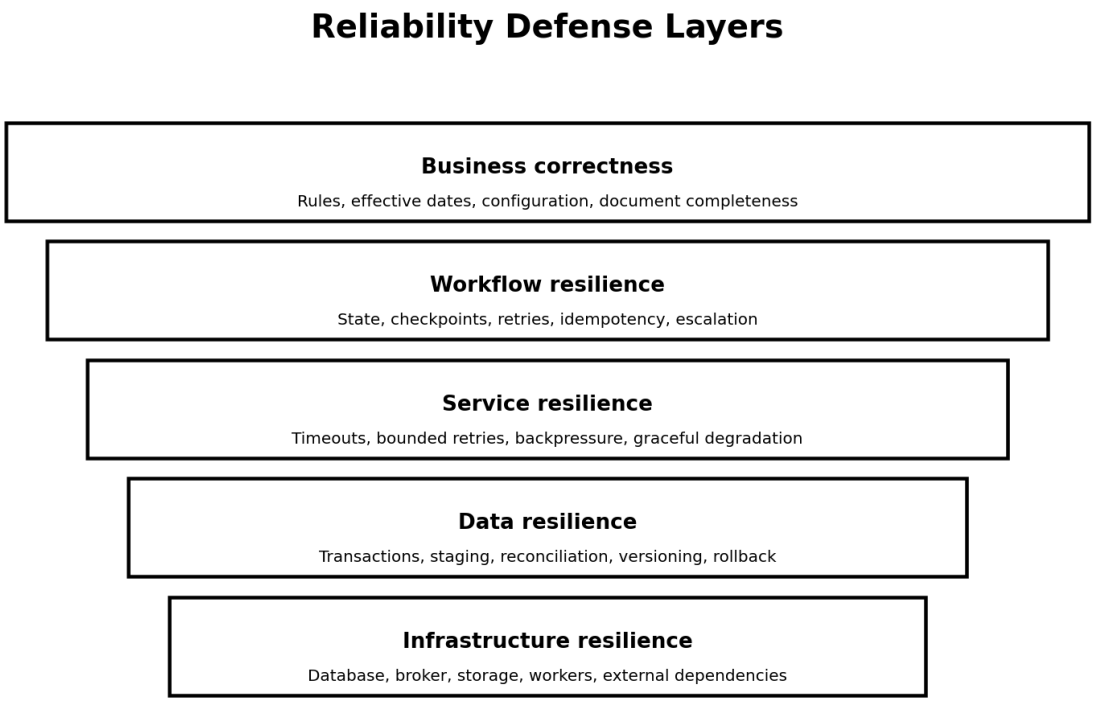


Figure 1. Reliability must be protected from infrastructure through business outcome.

2. Reliability Context and Scope

The represented systems have different failure characteristics. The annual income-limit workflow is batch-oriented and data-intensive. Eligibility and rent calculations are transactional and rule-sensitive. RabbitMQ workloads are asynchronous and vulnerable to duplicate delivery. Document generation depends on templates, data, rendering, and storage. Migrations can leave code and data in incompatible states. LeaseI and AgentProbe introduce probabilistic model behavior and external tools.

System	Critical failure	Required recovery outcome
Annual income-limit import	Partial or incorrect publication	Restore a validated version and reconcile downstream use
Eligibility and financial logic	Wrong result from rule, date, data, or setting	Reproduce the decision and correct the authoritative input or logic
RabbitMQ processing	Backlog, redelivery, poison message, duplicate effect	Resume processing without repeated business impact
Document generation	Blank fields, incompatible mapping, storage failure	Regenerate the correct version and preserve affected-document evidence
Database migration	Partial schema/data state or disputed execution	Contain changes, validate checkpoints, roll back or complete safely
External APIs and S3	Timeout, stale response, unavailable object	Fail safely, retry selectively, and preserve user-visible status
LeaseI	Unsupported root cause or failed tool	Return evidence or escalate rather than invent a conclusion
AgentProbe	Regression missed before deployment	Block release or surface the specific failure category

3. Reliability Principles

Business correctness is the objective

Infrastructure availability is necessary but insufficient. Reliability is measured by correct, reproducible customer outcomes.

Recovery is designed before failure

Runbooks, checkpoints, rollback, and validation should exist before a production incident creates urgency.

Retries are bounded and selective

Only transient failures should be retried automatically. Permanent and unknown failures need isolation or human review.

Every side effect must tolerate redelivery

Asynchronous systems require idempotency because delivery can occur more than once.

State must be explicit

Long-running work needs durable states that distinguish requested, running, validating, completed, failed, and escalated.

Failure should be isolated

One bad property, document, message, or jurisdiction should not corrupt unrelated work.

Evidence is preserved

Recovery should not destroy the data, logs, versions, or timestamps needed to identify the root cause.

Humans remain in control of high-impact correction

Regulated outcomes and production changes need accountable review when evidence is incomplete or conflicting.

4. Failure Taxonomy

Failure class	Examples	Default response
Transient	Temporary network failure, broker connection loss, external API timeout	Retry with bounded backoff and jitter
Permanent input failure	Invalid schema, missing required mapping, unsupported document field	Reject or quarantine with actionable evidence
Business-rule failure	Ineligible household, invalid state transition, threshold violation	Return deterministic result; do not retry
Concurrency failure	Lock timeout, duplicate worker, version conflict	Retry only when operation is idempotent and conflict policy is clear
Dependency degradation	Slow database, rate-limited API, model unavailable	Timeout, shed load, degrade capability, or queue work
Partial completion	Some rows, documents, or migration stages succeed	Use checkpoints, reconciliation, compensation, or targeted replay
Unknown or conflicting evidence	Unexpected production state, inconsistent sources	Contain and escalate; preserve evidence before change
Security or authorization failure	Expired credential, denied tenant access, invalid tool request	Fail closed and do not retry blindly

Correct classification prevents harmful automation. Retrying an invalid business request wastes capacity. Treating an external timeout as a permanent failure may create unnecessary manual work. Treating an unknown production state as transient may repeat a destructive action.

5. Reliability Architecture Layers

Reliability controls should be distributed across the architecture. A queue cannot compensate for invalid business logic, and a database transaction cannot guarantee that an external document object exists. Each layer protects a specific type of failure.

- Infrastructure: health, redundancy, connection management, durable storage, broker availability.
- Service: timeouts, circuit breaking, bounded retries, validation, rate control, graceful shutdown.
- Workflow: durable state, checkpoints, idempotency, replay, human escalation.
- Data: staging, transactions, versioning, effective dates, reconciliation, backups.
- Business: deterministic rules, configuration precedence, output validation, auditability.

6. Workflow State and Recovery

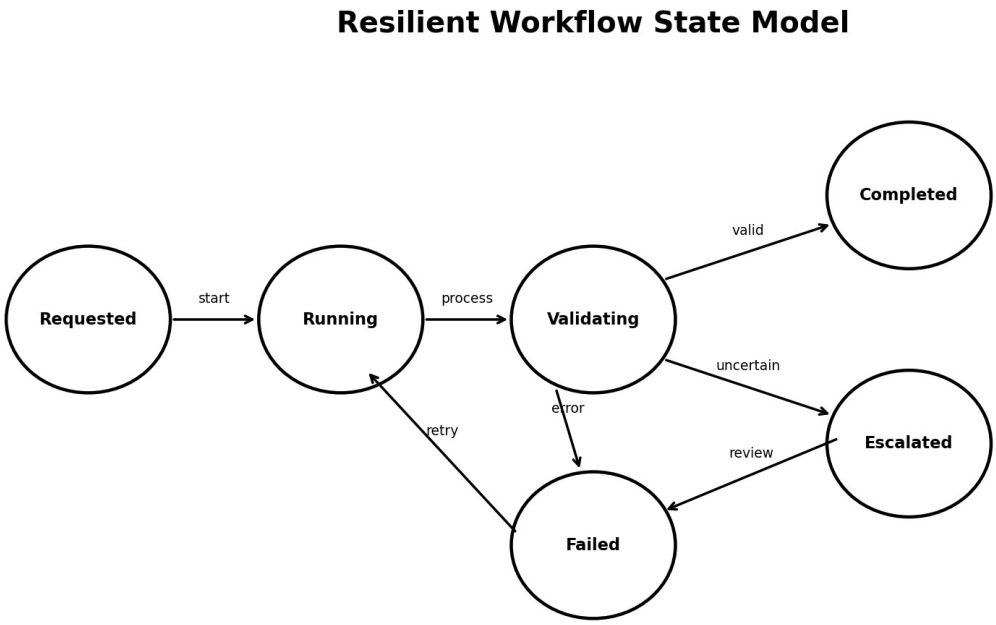


Figure 2. Explicit states make recovery and escalation predictable.

A long-running import, migration, document batch, or AI investigation should not be represented by a single Boolean flag. Durable state helps operators answer whether work has started, which step completed, whether retry is safe, and whether manual review is required.

State	Meaning	Allowed next action
Requested	Work accepted but not started	Start or cancel
Running	A worker owns active execution	Continue, checkpoint, or fail
Validating	Processing finished; business checks pending	Publish, retry validation, or escalate
Completed	All technical and business checks passed	Retain evidence and close
Failed - retryable	Transient failure with safe retry path	Retry after backoff

State	Meaning	Allowed next action
Failed - terminal	Permanent input or invariant failure	Correct input or close
Escalated	Evidence incomplete or decision high impact	Human review and explicit resolution
Rolled back	Previous valid state restored	Investigate before reattempt

7. Database Reliability

Relational databases are the source of truth for many of the represented systems. Reliability controls must protect transaction integrity, performance, historical correctness, and recovery.

- Use transactions to align changes that must succeed or fail together.
- Avoid long transactions that create blocking and expand rollback cost.
- Define affected-row expectations for production scripts.
- Use effective-dated records rather than destructive in-place replacement when history matters.
- Preserve migration checkpoints, pre-change counts, and post-change reconciliation.
- Make critical scripts repeatable or guarded against accidental rerun.
- Treat backups and dumps as recovery evidence, not as a substitute for validation.

The Version 2 migration incident demonstrated that database evidence can resolve disagreement. Timestamps and dumps helped distinguish when data changed from when an issue was observed. That is a reliability practice because it prevents the wrong corrective action.

8. ETL and Data Publication Reliability

Annual HUD data processing transforms a few thousand source rows into approximately 180,000 domain records. Reliability depends on isolating ingestion from publication and validating every transformation stage.

Stage	Reliability control
Source acquisition	Version, checksum, retrieval date, source-year validation
Parsing	Schema checks, malformed-row capture, required-field validation
Staging	Duplicate detection, mapping validation, no downstream visibility
Transformation	Deterministic procedure version, counts, ranges, expansion checks
Validation	Representative business scenarios and reconciliation
Publication	Atomic activation or controlled version switch
Post-publication	Eligibility, rent, tax-credit, HAP, and document spot checks

Stage	Reliability control
Recovery	Restore prior active version or correct and republish from staged evidence

A critical design principle is that partially transformed data should not become visible to downstream calculations. Staging and publication boundaries reduce the chance that customers observe an inconsistent mixed state.

9. RabbitMQ and Asynchronous Processing

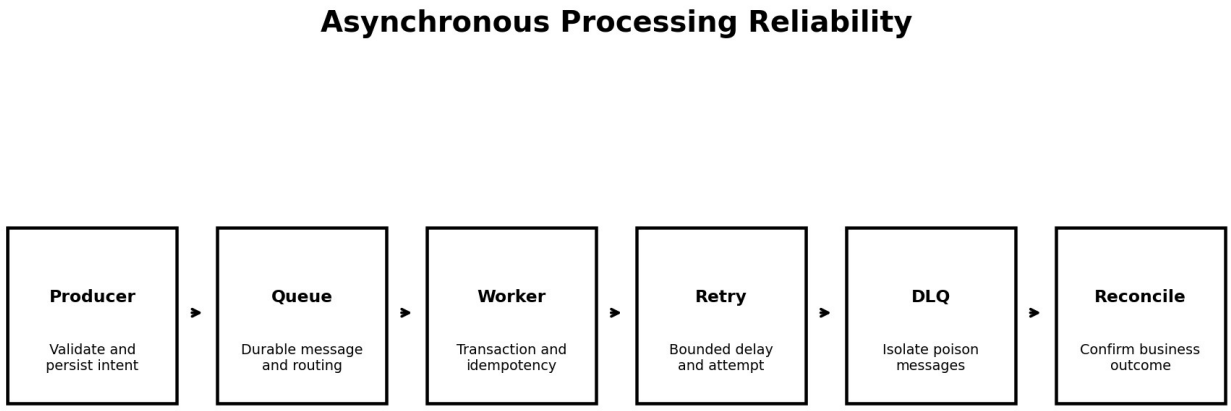


Figure 3. Reliable asynchronous processing combines durable delivery with safe business execution.

RabbitMQ decouples large annual gross-rent work from the user request, but the broker only provides delivery mechanics. The application must define message contracts, transaction boundaries, acknowledgement behavior, retries, dead-letter handling, and business reconciliation.

Component	Reliability responsibility
Producer	Validate intent, create stable business identifier, publish only after required state is durable
Exchange and queue	Durable routing, explicit binding, capacity monitoring
Consumer	Validate contract, acquire authoritative data, enforce idempotency
Business transaction	Commit all required database effects consistently
Acknowledgement	Acknowledge only after durable success
Retry path	Classify transient error, delay, cap attempts
Dead-letter path	Preserve evidence and stop infinite processing

Component	Reliability responsibility
Reconciliation	Confirm all expected entities reached a terminal business state

10. Idempotency and Duplicate Prevention

At-least-once delivery means a message may be delivered more than once. Idempotency ensures that repeated execution produces one intended business effect.

- Use a stable idempotency key derived from the business operation, not a random retry identifier.
- Record processing state or a unique constraint in the same transactional boundary as the business effect.
- Return the prior successful result when the same operation is repeated.
- Design document generation and external writes with versioned object keys or deduplication records.
- Include operation version when a legitimate reprocessing must produce a new result.
- Reconcile final business records rather than assuming acknowledgements prove correctness.

Idempotency is especially important for financial and rent-related updates. A duplicated message should not produce duplicated rent schedules or repeated downstream documents.

11. Retry, Timeout, and Backoff Strategy

Control	Purpose	Failure if missing
Timeout	Bound how long a dependency can block work	Thread or worker exhaustion
Retry classification	Retry only transient failures	Repeated permanent errors and wasted capacity
Exponential backoff	Reduce pressure during dependency recovery	Retry storm
Jitter	Prevent synchronized retries	Thundering herd
Attempt cap	Move unrecoverable work to review	Infinite loops
Retry budget	Limit total dependency amplification	System-wide cascading failure
Idempotency	Make repeated execution safe	Duplicate business effects

Retries belong at the layer that understands the operation. A low-level HTTP client can retry a safe GET, but a business service must decide whether retrying a write is safe. Multiple nested retry layers can multiply traffic unexpectedly and should be coordinated.

12. Dead-Letter and Poison-Message Handling

A dead-letter queue is not a complete recovery strategy. It is an isolation mechanism. Teams still need ownership, classification, correction, replay rules, and closure evidence.

1. Capture message identifier, business context, attempt history, safe error category, and relevant release version.
2. Determine whether the cause is invalid input, code defect, unavailable dependency, stale configuration, or data inconsistency.
3. Correct the underlying cause before replay.
4. Replay through the normal validated path rather than editing business state manually where possible.
5. Verify the business outcome and close or archive the dead-letter record.
6. Add a regression test, validation, monitor, or documentation update.

13. API and Service Resilience

Interactive APIs should fail quickly and clearly rather than tying up resources during dependency degradation. Long-running work should move to asynchronous execution when the user does not require an immediate final result.

- Validate requests before consuming expensive resources.
- Set explicit connection and response timeouts.
- Use structured errors that distinguish validation, authorization, dependency, and internal failures.
- Apply backpressure, rate limiting, and concurrency limits to protect shared dependencies.
- Use pagination and bounded response sizes.
- Return an operation identifier for long-running work and expose durable status.
- Make shutdown graceful so in-flight work either completes or becomes recoverable.

14. External Dependency Failure

The represented systems depend on APIs, object storage, databases, message brokers, and model providers. Each dependency needs a failure policy based on freshness, authority, and business impact.

Dependency	Safe behavior during failure
HUD income-limit API	Use no guessed value; return controlled unavailability or a governed cached version with source and date
AWS S3	Retry transient operations; do not mark document complete until object integrity and metadata are confirmed
Database	Reject or queue work if authoritative state cannot be read safely
RabbitMQ	Persist intent or fail the request explicitly; avoid silent message loss
Model provider	Degrade to deterministic/retrieval-only behavior or escalate
Configuration service	Fail closed for high-impact rules unless a versioned approved snapshot is permitted

15. Document Generation and Storage Recovery

Document generation is a multi-step workflow: select template, resolve fields, retrieve data, render, store, and expose the object. Reliability requires a status model and versioned evidence for each step.

Failure	Recovery approach
Missing merge field	Fail validation or mark incomplete; correct mapping and regenerate
Template incompatibility	Use compatible version or rollback template activation
Data retrieval failure	Retry only if transient; preserve requested document context
Renderer failure	Retry on another worker when safe; isolate repeated template-specific defects
Object write failure	Do not publish metadata as complete; retry and verify checksum
Incorrect generated batch	Identify affected version and regenerate only impacted documents
Access metadata mismatch	Block delivery and correct ownership before release

The six urgent defects after the merge-field migration demonstrated that recovery must include customer-facing document regeneration, not only a code fix. The system needs to identify which outputs were generated with the defective mapping.

16. Release, Migration, and Rollback Reliability

A reliable deployment process treats rollback as a state-management problem. Application code, database schema, transformed data, queued messages, generated documents, and external objects may all require different recovery actions.

Release concern	Reliability design
Compatibility	Use additive schema changes and staged code where possible
Data transformation	Checkpoint and reconcile each phase
Go/no-go	Use named validation gates and evidence
Rollback trigger	Define business and technical thresholds before deployment
Rollback scope	Specify code, schema, data, queue, document, and cache actions
Post-rollback	Verify the previous state is actually restored

Release concern	Reliability design
Audit	Retain scripts, results, decisions, and deviations

Phased rollout reduces blast radius and improves diagnosis. It should be preferred when the change can be partitioned by property, tenant, document type, jurisdiction, worker group, or feature flag.

17. Configuration and Feature-Flag Reliability

Configuration changes runtime behavior and must be designed with the same discipline as code. The nearly 24-hour eligibility incident showed how an unknown system-level setting can override correct module logic.

- Define deterministic precedence across global, module, program, and property scopes.
- Record old value, new value, actor, reason, release, and effective time.
- Validate the resolved configuration, not only the stored record.
- Use staged rollout and immediate disable capability for risky flags.
- Monitor drift and stale overrides.
- Retire temporary flags after rollout to reduce hidden branches.
- Surface configuration dependencies in support and engineering tools.

18. Graceful Degradation and Partial Availability

Graceful degradation means preserving safe core behavior when an optional capability fails. It does not mean returning fabricated or stale regulated answers.

Failure	Acceptable degradation
Historical ticket search unavailable	Continue with product rules and live tools; disclose missing historical evidence
Model unavailable	Return deterministic findings and escalation packet without generated narrative
HUD live API unavailable	Use an explicitly governed cached value or mark the lookup unavailable
Document preview unavailable	Preserve generation request and expose pending status; do not claim completion
Analytics dashboard unavailable	Keep core transaction path operating while telemetry recovers
One batch item fails	Isolate the item and continue unrelated work when business rules permit

19. Human Escalation and Operational Control

Some failures cannot be resolved safely by automatic retry. Unknown state, conflicting evidence, high-impact configuration, regulated decisions, and repeated poison messages require human review.

Escalation packet element	Purpose
Issue summary	Explain customer impact and expected behavior
Workflow state	Show completed, failed, and pending steps
Evidence	Include rules, values, settings, versions, logs, and tool responses
Actions attempted	Prevent repeated blind retries
Hypotheses	Separate supported, contradicted, and unknown causes
Containment	Document what stopped further impact
Recommended next step	Guide the accountable engineer or domain expert
Approval and closure	Record the final decision and validation

20. AI Runtime Reliability

AI reliability is not equivalent to model availability. The system must retrieve authorized evidence, execute approved tools, validate claims, handle uncertainty, and escalate when the information is insufficient.

- Keep deterministic business logic as the source of truth.
- Treat retrieved documents and tickets as untrusted data, not instructions.
- Use allowlisted read-only tools with validated parameters.
- Record retrieval and tool failures separately from model failures.
- Require evidence for important claims.
- Expose missing or conflicting sources.
- Avoid model self-confidence as a reliability signal.
- Enforce human review for eligibility, configuration, and calculation outcomes.

LeaseI currently uses fixed graph routing and hand-tuned confidence constants. A production target would use structured tool contracts, stronger retrieval, measured evidence signals, and more comprehensive safety evaluation.

21. AI Evaluation and Regression Protection

AgentProbe reflects the principle that reliability must be measured before deployment. Prompt, model, retrieval, chunking, and tool changes can create regressions even when the application code still runs.

Evaluation dimension	Reliability question
Task success	Does the workflow solve the intended scenario?
Retrieval	Did it find the correct authorized evidence?
Grounding	Are conclusions supported by sources?
Tool use	Were the correct tools and parameters used?

Evaluation dimension	Reliability question
Safety	Did the system avoid regulated auto-decisions and unauthorized actions?
Robustness	Does behavior hold under paraphrase, missing data, conflict, and dependency failure?
Performance	Are latency and cost within operational limits?
Regression	Is the new version no worse than the approved baseline on critical cases?

22. Observability and Reliability Metrics

Area	Reliability metrics
API	Success rate, p95/p99 latency, timeouts, rejected requests
Database	Query latency, lock waits, deadlocks, transaction age, reconciliation variance
ETL	Stage duration, invalid records, expansion ratio, publication failure
RabbitMQ	Oldest message age, retry rate, DLQ depth, processing duration, duplicate suppression
Documents	Generation success, missing-field rate, storage integrity, regeneration count
Migrations	Checkpoint completion, affected rows, validation failures, rollback use
AI	Retrieval empty rate, tool failure, unsupported claim, escalation, safety-test failure
Recovery	Mean time to detect, contain, recover, and prevent recurrence

Metrics should be connected to runbooks and ownership. A reliability signal has limited value if no one knows which safe action it should trigger.

23. Testing Failure and Recovery

- Inject timeouts and unavailable dependencies.
- Deliver duplicate and out-of-order messages.
- Simulate worker termination before and after commit.
- Test migration interruption at every checkpoint.
- Verify retry caps and dead-letter routing.
- Test invalid data, stale effective dates, and conflicting configuration.
- Verify document regeneration and affected-output identification.

- Test rollback of code and state, not only deployment artifacts.
- Test AI behavior with empty retrieval, tool failure, contradictory sources, and prompt injection.
- Confirm observability and audit evidence during failure paths.

A recovery test is successful only when the final business state is correct and the evidence explains what happened. Restarting a process is not sufficient.

24. Incident Recovery Examples

24.1 Hidden configuration dependency

A customer failed Affordable Housing eligibility even though the domain rules and values were correct. The root cause was a system-level setting changed by another team. The incident demonstrates the need for configuration history, cross-module dependency visibility, read-only investigation, and escalation when evidence crosses ownership boundaries.

24.2 Version 2 database rollout

A migration was initially believed to be incorrect. Database dumps, timestamps, and phased validation showed that the observed state did not prove a faulty migration. Preserving evidence prevented an unnecessary corrective action and reinforced the value of checkpointed releases and evidence-based recovery.

24.3 Merge-field migration defects

Six urgent defects appeared after a document mapping change. Recovery required identifying the affected templates and fields, correcting mappings, validating old and new documents, and regenerating impacted customer outputs. The lesson was that document reliability includes output lineage and targeted regeneration.

25. Reliability Trade-offs

Synchronous simplicity versus asynchronous resilience

Synchronous work is easier to reason about, while queues improve isolation and scalability but require state, idempotency, and operations.

Automatic retry versus controlled failure

Retries reduce manual work for transient faults but amplify permanent defects when classification is weak.

Strict transactions versus throughput

Large transactions simplify atomicity but increase locking and rollback cost. Chunked workflows improve scale but require checkpoints and reconciliation.

Availability versus correctness

Serving a guessed or stale regulated answer may be worse than controlled unavailability.

Generic infrastructure versus domain controls

Shared platforms reduce duplication, but domain-specific validation is required to detect semantically wrong outcomes.

Fast recovery versus evidence preservation

Immediate changes may restore service but destroy root-cause evidence. Containment and evidence capture should precede correction when feasible.

26. Modernization Roadmap

Phase	Focus	Expected outcome
1. Standardize workflow state	Durable states, correlation, terminal outcomes	Recoverable long-running work
2. Strengthen idempotency	Business keys, transactional deduplication, reconciliation	Safe retries and redelivery
3. Formalize failure policy	Timeouts, retry classification, backoff, DLQs	Reduced retry storms and hidden failure
4. Improve release recovery	Checkpointed migrations, state-aware rollback, phased rollout	Lower production blast radius
5. Add cross-module reliability	Configuration history, dependency contracts, document lineage	Faster root-cause isolation
6. Mature AI reliability	Tool schemas, measured confidence, safety gates, evaluation baselines	Trustworthy agentic workflows
7. Automate incident learning	Incident-to-test, alert, runbook, and knowledge update	Reduced recurrence

27. Reusable Reliability Checklists

27.1 New API or service

- Define timeout and dependency failure behavior.
- Identify which operations are safe to retry.
- Add idempotency for repeated writes.
- Define backpressure and concurrency limits.
- Expose structured status and errors.
- Instrument business and technical outcomes.
- Write a recovery runbook before high-risk launch.

27.2 Batch, ETL, or migration

- Use staging and explicit publication.
- Persist checkpoints and workflow state.

- Validate counts, mappings, ranges, and effective dates.
- Bound transaction and batch size.
- Define restart, replay, and rollback behavior.
- Reconcile downstream outcomes.
- Preserve source, version, and validation evidence.

27.3 RabbitMQ workflow

- Use stable message and idempotency identifiers.
- Acknowledge only after durable success.
- Classify transient and permanent failure.
- Bound retries and configure dead-letter handling.
- Monitor oldest message age and poison-message patterns.
- Document safe replay and final reconciliation.

27.4 AI workflow

- Keep regulated decisions deterministic.
- Use authorized retrieval and allowlisted tools.
- Handle empty, stale, and conflicting evidence.
- Validate consequential claims.
- Measure quality, safety, latency, and cost.
- Escalate when evidence or confidence is insufficient.
- Run regression and failure-mode evaluation before release.

28. Lessons Learned

- Reliable systems recover to a validated business state, not merely a running process.
- Retries without idempotency and classification can make failure worse.
- Explicit workflow state is essential for long-running and multi-step work.
- Configuration, documents, queues, and external objects are part of recovery scope.
- Business reconciliation is stronger evidence than infrastructure success.
- Rollback must account for code, schema, data, messages, documents, and caches.
- Unknown or conflicting evidence should trigger containment and human escalation.
- AI reliability requires grounding, tools, validation, evaluation, and safety boundaries.
- Every production failure should improve a test, monitor, runbook, or durable knowledge source.

Closing Perspective

My strongest reliability lessons came from owning systems after deployment. Production behavior exposed hidden dependencies, migration risks, duplicate-processing concerns, document recovery needs, and the importance of evidence. That experience shaped how I approach both traditional software and AI: design failure paths early, preserve authoritative state, validate the business outcome, and make recovery understandable to the next engineer.